

[CI2] Concepts informatiques — Cours 10

Jérémy Ledent

Université Paris Cité, IRIF



INSTITUT
DE RECHERCHE
EN INFORMATIQUE
FONDAMENTALE



Université
Paris Cité

CC2 la semaine du 13 avril

- ▶ **Sujets :** Cours 8 et 9 / TD 8 et 9.
 - Récursivité
 - Récursivité terminale
 - Mémoïsation
- ▶ **Format :** contrôle d'1h sur votre créneau de TD.

Retours sur le partiel

Exercice « Échange »

```
static void exchange(int x, int y) {  
    int z = x;  
    x = y;  
    y = z;  
}  
  
static void exchange(int[] t, int[] u) {  
    int[] v = t;  
    t = u;  
    u = v;  
}
```

```
public static void main(String[] args){  
    int[] [] m = {{ 1, 2, 3 },  
                  { 4, 5, 6 },  
                  { 7, 8, 9 }};  
    exchange(m[0][0], m[2][2]);  
    exchange(m[1], m[2]);  
    exchange(m[0], m[1]);  
    exchange(m[0][2], m[2][0]);  
    exchange(m[2], m[0]);  
    exchange(m[0][0], m[1][1]);  
    System.out.println(m[0][0]);  
}
```

[\[Visualiser sur PythonTutor\]](#)

Exercice « Sous-typage »

On suppose qu'on a défini quatre fonctions, avec les en-têtes ci-dessous :

```
public static int f(char c)
public static char f(long n)
public static int f(int a, double b)
public static double f(double a, int b)
```

Parmi les instructions ci-dessous, lesquelles ne produisent **pas** d'erreur ?

- ▶ `int k = f(7, 3.14);`
- ▶ `char p = f(0.5);`
- ▶ `int x = f(12);`
- ▶ `char c = f(f(3));`
- ▶ `long n = f(f('a'));`
- ▶ `double t = f('b', 3.33);`
- ▶ `double u = f(4, 7);`
- ▶ `int m = f(f('#'), f(1.414, 8));`

Exercice « Traduction » — Que vaut le tableau à la fin de l'exécution ?

```
public static void h(int[] tab, int i) {  
    while (i != 0) {  
        tab[i] = tab[i] + i;  
        i--;  
    }  
}  
  
public static void main(String[] args) {  
    int[] t = new int[10];  
    for (int i = 0; i < 10; i++) {  
        h(t, i++);  
    }  
}
```

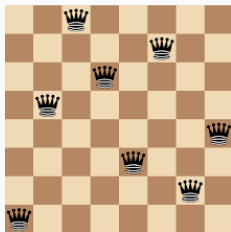
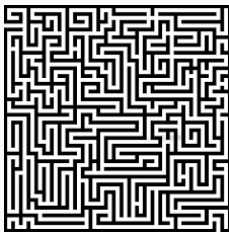
[\[Visualiser sur PythonTutor\]](#)

Backtracking

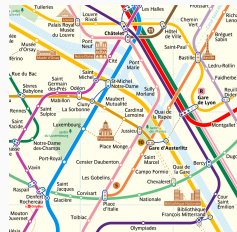
Backtracking

Le **backtracking** (en Français, *retour sur trace*) est une technique qui permet d'**explorer de manière exhaustive** toutes les solutions d'un problème.

Contrairement à un algorithme de type « brute-force », on construit la solution de manière **incrémentale**; et on revient en arrière dès qu'on détecte qu'une solution (partielle) ne peut pas aboutir.



5	3			7			
6			1	9	5		
	9	8				6	
8				6			3
4			8		3		1
7				2			6
	6					2	8
			4	1	9		5
				8			7
							9



Comparaison : Brute-force vs Backtracking

La méthode **brute-force** consiste à énumérer toutes les solutions possibles à un problème, et les essayer une à une.

Pour un Sudoku :

	5	7		6			1	
8		6	2		5			9
	4							6
	2	4	3	5				
				1	2			5
5	3						7	2
			1			9	6	
			6					
1	6	9	5	7	3			

Comparaison : Brute-force vs Backtracking

La méthode **brute-force** consiste à énumérer toutes les solutions possibles à un problème, et les essayer une à une.

Pour un Sudoku :

- 1, 1, 1, 1, ..., 1, 1 ne marche pas. ✗

1	5	7	1	6	1	1	1	1
8	1	6	2	1	5	1	1	9
1	4	1	1	1	1	1	1	6
1	2	4	3	5	1	1	1	1
1	1	1	1	1	2	1	1	5
5	3	1	1	1	1	1	7	2
1	1	1	1	1	1	9	6	1
1	1	1	6	1	1	1	1	1
1	6	9	5	7	3	1	1	1

Comparaison : Brute-force vs Backtracking

La méthode **brute-force** consiste à énumérer toutes les solutions possibles à un problème, et les essayer une à une.

Pour un Sudoku :

- ▶ 1, 1, 1, 1, ..., 1, 1 ne marche pas. ✗
- ▶ 1, 1, 1, 1, ..., 1, 2 ne marche pas. ✗

1	5	7	1	6	1	1	1	1
8	1	6	2	1	5	1	1	9
1	4	1	1	1	1	1	1	6
1	2	4	3	5	1	1	1	1
1	1	1	1	1	2	1	1	5
5	3	1	1	1	1	1	7	2
1	1	1	1	1	1	9	6	1
1	1	1	6	1	1	1	1	1
1	6	9	5	7	3	1	1	2

Comparaison : Brute-force vs Backtracking

La méthode **brute-force** consiste à énumérer toutes les solutions possibles à un problème, et les essayer une à une.

Pour un Sudoku :

- ▶ 1, 1, 1, 1, ..., 1, 1 ne marche pas. ✗
- ▶ 1, 1, 1, 1, ..., 1, 2 ne marche pas. ✗
- ▶ 1, 1, 1, 1, ..., 1, 3 ne marche pas. ✗
- ▶ etc...

1	5	7	1	6	1	1	1	1
8	1	6	2	1	5	1	1	9
1	4	1	1	1	1	1	1	6
1	2	4	3	5	1	1	1	1
1	1	1	1	1	2	1	1	5
5	3	1	1	1	1	1	7	2
1	1	1	1	1	1	9	6	1
1	1	1	6	1	1	1	1	1
1	6	9	5	7	3	1	1	3

Comparaison : Brute-force vs Backtracking

La méthode **brute-force** consiste à énumérer toutes les solutions possibles à un problème, et les essayer une à une.

Pour un Sudoku :

- ▶ 1, 1, 1, 1, ..., 1, 1 ne marche pas. ✗
- ▶ 1, 1, 1, 1, ..., 1, 2 ne marche pas. ✗
- ▶ 1, 1, 1, 1, ..., 1, 3 ne marche pas. ✗
- ▶ etc...

Sur l'exemple, 49 cases à remplir, 9 chiffres possibles par case.

Total : 9^{49} combinaisons possibles.

1	5	7	1	6	1	1	1	1
8	1	6	2	1	5	1	1	9
1	4	1	1	1	1	1	1	6
1	2	4	3	5	1	1	1	1
1	1	1	1	1	2	1	1	5
5	3	1	1	1	1	1	7	2
1	1	1	1	1	1	9	6	1
1	1	1	6	1	1	1	1	1
1	6	9	5	7	3	1	1	3

Comparaison : Brute-force vs Backtracking

La méthode du **backtracking** construit la solution case par case, et s'arrête dès qu'une contradiction est trouvée.

	5	7		6			1	
8		6	2		5			9
	4							6
	2	4	3	5				
				1	2			5
5	3						7	2
			1			9	6	
			6					
1	6	9	5	7	3			

Comparaison : Brute-force vs Backtracking

La méthode du **backtracking** construit la solution case par case, et s'arrête dès qu'une contradiction est trouvée.

- 1ère case : possibles = {**2**, 3, 9}

2	5	7		6			1	
8		6	2		5			9
	4							6
	2	4	3	5				
				1	2			5
5	3						7	2
			1			9	6	
			6					
1	6	9	5	7	3			

Comparaison : Brute-force vs Backtracking

La méthode du **backtracking** construit la solution case par case, et s'arrête dès qu'une contradiction est trouvée.

- ▶ 1ère case : possibles = {**2**, 3, 9}
- ▶ 2ème case : possibles = {**4**, 8, 9}

2	5	7	4	6			1	
8		6	2		5			9
	4							6
	2	4	3	5				
				1	2			5
5	3						7	2
			1			9	6	
			6					
1	6	9	5	7	3			

Comparaison : Brute-force vs Backtracking

La méthode du **backtracking** construit la solution case par case, et s'arrête dès qu'une contradiction est trouvée.

- ▶ 1ère case : possibles = {2, 3, 9}
- ▶ 2ème case : possibles = {4, 8, 9}
- ▶ 3ème case : possibles = {8, 9}

2	5	7	4	6	8		1	
8		6	2		5			9
	4							6
	2	4	3	5				
				1	2			5
5	3						7	2
			1			9	6	
			6					
1	6	9	5	7	3			

Comparaison : Brute-force vs Backtracking

La méthode du **backtracking** construit la solution case par case, et s'arrête dès qu'une contradiction est trouvée.

- ▶ 1ère case : possibles = {**2**, 3, 9}
- ▶ 2ème case : possibles = {**4**, 8, 9}
- ▶ 3ème case : possibles = {**8**, 9}
- ▶ 4ème case : possibles = {**3**}

2	5	7	4	6	8	3	1	
8		6	2		5			9
	4							6
	2	4	3	5				
				1	2			5
5	3						7	2
			1			9	6	
			6					
1	6	9	5	7	3			

Comparaison : Brute-force vs Backtracking

La méthode du **backtracking** construit la solution case par case, et s'arrête dès qu'une contradiction est trouvée.

- ▶ 1ère case : possibles = {2, 3, 9}
- ▶ 2ème case : possibles = {4, 8, 9}
- ▶ 3ème case : possibles = {8, 9}
- ▶ 4ème case : possibles = {3}
- ▶ 5ème case : perdu!

Il faut revenir en arrière (« backtracker »).

2	5	7	4	6	8	3	1	
8		6	2		5			9
	4							6
	2	4	3	5				
				1	2			5
5	3						7	2
			1			9	6	
			6					
1	6	9	5	7	3			

Le cœur du backtracking : faire des choix

Un problème de backtracking consiste à faire une succession de **choix** :

- ▶ **Labyrinthe** : à chaque intersection, choisir le chemin à prendre.
- ▶ **Sudoku** : dans chaque case vide, choisir un nombre entre 1 et 9.



Le cœur du backtracking : faire des choix

- En général, on n'a aucun moyen de savoir à l'avance quel choix est le bon. Notre objectif est de tous les essayer de façon **systematique**.

Le cœur du backtracking : faire des choix

- ▶ En général, on n'a aucun moyen de savoir à l'avance quel choix est le bon. Notre objectif est de tous les essayer de façon **systématique**.
- ▶ Parfois, on se retrouve dans une **impasse** : il n'y a plus aucun choix possible. Ça veut dire que l'un des choix précédents était mauvais.

Le cœur du backtracking : faire des choix

- ▶ En général, on n'a aucun moyen de savoir à l'avance quel choix est le bon. Notre objectif est de tous les essayer de façon **systématique**.
- ▶ Parfois, on se retrouve dans une **impasse** : il n'y a plus aucun choix possible. Ça veut dire que l'un des choix précédents était mauvais.
- ▶ Dans ce cas, on revient en arrière pour essayer un autre choix. D'où le nom de « **backtracking** ».

Schéma général de backtracking

Au cours de l'algorithme, on construit une **solution partielle**. Par exemple :

- ▶ Pour un labyrinthe, le chemin parcouru jusqu'ici.
- ▶ Pour un sudoku, les valeurs des cases déjà remplies.

Schéma général de backtracking

Au cours de l'algorithme, on construit une **solution partielle**. Par exemple :

- ▶ Pour un labyrinthe, le chemin parcouru jusqu'ici.
- ▶ Pour un sudoku, les valeurs des cases déjà remplies.

À chaque étape de l'algorithme :

1. Si la solution est complète, on a terminé.
2. Sinon, on calcule la liste des choix possibles à partir de la solution courante.
3. Pour chacun des choix possibles :
 - On l'ajoute à la solution partielle.
 - On fait un **appel récuratif** pour résoudre le problème à partir de ce nouvel état.
 - Si aucune solution trouvée, on backtrack et on teste le prochain choix possible.
4. Si aucun des choix possibles n'a abouti : aucune solution trouvée.

Questions à se poser

Avant de se lancer dans le code, il faut se poser les bonnes questions :

- ▶ C'est quoi une solution partielle au problème?
Quelle **structure de données** utiliser pour la représenter?

Questions à se poser

Avant de se lancer dans le code, il faut se poser les bonnes questions :

- ▶ C'est quoi une solution partielle au problème?
Quelle **structure de données** utiliser pour la représenter?
- ▶ Comment on sait quand une solution est complète?

Questions à se poser

Avant de se lancer dans le code, il faut se poser les bonnes questions :

- ▶ C'est quoi une solution partielle au problème?
Quelle **structure de données** utiliser pour la représenter?
- ▶ Comment on sait quand une solution est complète?
- ▶ À une étape donnée, comment calculer la liste des choix possibles?
Souvent, on peut décomposer cette étape en deux :
 - Quels sont les **choix potentiels**? (Sudoku : tous les chiffres de 1 à 9)
 - Parmi eux, lesquels sont **valides**? (Sudoku : vérifier les lignes/colonnes/carrés)

Variante 1 — Énumérer toutes les solutions

```
public static void resoudreParBacktracking() {  
    if (solutionComplete()){  
        System.out.println("Solution trouvée : " + solution);  
    } else {  
        for (Choix c : choixPossibles()) {  
            solution.add(c);           // On essaye le choix c  
            resoudreParBacktracking(); // Résoudre la suite du problème  
            solution.remove(c);        // On backtrack  
        }  
    }  
}
```

Les différentes fonctions sont à personnaliser suivant les cas.

Variante 2 — S'arrêter à la première solution trouvée

```
public static boolean resoudreParBacktracking() {  
    if (solutionComplete()){  
        return true;  
    }  
    for (Choix c : choixPossibles()) {  
        solution.add(c);                // On essaye le choix c  
        if (resoudreParBacktracking()) { // Résoudre la suite du problème  
            return true;                // Si une solution est trouvée,  
        }                               // on s'arrête tout de suite  
        solution.remove(c);             // Sinon, on backtrack  
    }  
    return false; // Aucune solution n'a été trouvée  
}
```

Le schéma général explique le principe du backtracking, ce n'est **pas** une recette magique qui doit être suivie à la lettre!

- ▶ Pas d'apprentissage par cœur!
- ▶ L'important est de **comprendre** comment ça marche.
- ▶ Afin de pouvoir **adapter** à différents problèmes.

Pour l'instant tout ça est un peu vague, faisons des exemples concrets.

Exemple 1 :

Résolution d'un labyrinthe

Objectif

On dispose d'une classe `Labyrinthe.java`, qui contient :

- ▶ Des méthodes pour charger un labyrinthe à partir d'un fichier, et l'afficher.
- ▶ Une méthode `public void deplacer(char dir)` qui prend en entrée une direction (un caractère parmi `'H'`, `'B'`, `'G'`, `'D'`), et permet de se déplacer vers le haut, bas, gauche ou droite.

On a aussi une classe `Case.java`, qui représente une case du labyrinthe :

- ▶ une case est donnée par ses coordonnées `int x` et `int y`.
- ▶ Une méthode `public Case voisin(char dir)` qui renvoie la case voisine dans une direction donnée.

Objectif : dans `Main.java`, écrire une fonction qui résout le labyrinthe en utilisant la technique du backtracking.

Indications

- ▶ Une solution partielle, c'est la suite des déplacements déjà effectués.

- ▶ Il sera utile de marquer les cases qui ont déjà été visitées, pour éviter de tourner en rond. La classe `Labyrinthe` possède des méthodes `public void poserUnCaillou()` et `public void enleverUnCaillou()`.

- ▶ Le labyrinthe est représenté par une matrice d'entiers `int[][] grille` :
 - 0 représente une case vide (un couloir)
 - 1 représente un mur
 - 2 représente l'entrée
 - 3 représente la sortie
 - 4 représente un caillou (case marquée comme visitée)

À vous de jouer!

Vous pouvez télécharger [sur la page Moodle du cours](#) un dossier contenant :

- ▶ La bibliothèque de dessin `StdDraw.java`, pas besoin de lire son contenu.
- ▶ Les fichiers `Labyrinthe.java`, `Case.java`, à lire mais pas modifier.
- ▶ Le fichier `Main.java` à compléter.

Si besoin, il y a aussi le fichier `MainSolution.java` avec la solution faite en CM.

Mais je vous encourage à essayer de le refaire par vous-mêmes!

Exemple 2 :

Résolution d'un sudoku

Règles du Sudoku

Une grille de sudoku est un carré de 9×9 cases.

Le but est de remplir toutes les cases avec des chiffres compris entre 1 et 9, tels que :

- ▶ Chaque ligne contient tous les entiers de 1 à 9 sans répétition.
- ▶ Pareil pour chaque colonne.
- ▶ Pareil pour chacune des 9 sous-grilles carrées de taille 3×3 .

	5	7		6			1	
8		6	2		5			9
	4							6
	2	4	3	5				
				1	2			5
5	3						7	2
			1			9	6	
			6					
1	6	9	5	7	3			

Sur [la page Moodle du cours](#), vous trouverez un dossier contenant :

- ▶ Un fichier `Sudoku.java` qui contient des fonctions pour charger une grille de sudoku depuis un fichier texte, et l'afficher.
- ▶ Deux exemples de sudoku :
 - `sudoku1.txt` est de niveau moyen (d'après [sudoku.com](#)), alors que
 - `sudoku2.txt` est de niveau « extrême » (bon courage pour le faire à la main)

Objectif : dans `Sudoku.java`, écrire une fonction pour résoudre un sudoku.

Bonus : compter le nombre d'appels récurifs, comparer les deux exemples.

La solution du CM est donnée dans `SudokuSolution.java`, si besoin.

- ▶ C'est quoi une solution partielle?

- ▶ Comment déterminer quand la solution est complète?

- ▶ Comment déterminer les prochains choix possibles?

-
-
-
-